UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

| | | |
|---|---|---|
| COURSE | : | COMPUTER ORGANIZATION AND ARCHITECTURE |
| COURSE CODE | : | CSC429 |
| EXAMINATION | : | NOVEMBER 2026 (MOCK) |
| TIME | : | 3 HOURS |

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

*This marking scheme consists of 10 printed pages*

© Hak Cipta Universiti Teknologi MARA

# PART A — MARKING GUIDE

## QUESTION A1 (5 marks)

*Topic: Parallel and multiprocessor architecture*

*Source: CSC429 (8).pdf, page(s) 6*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) Classification of parallel processors is constructed according to the relationship between the instruction stream and the data stream, as proposed in Flynn's taxonomy. A parallel processor is described by whether it issues one or many instruction streams and whether it operates on one or many data streams, giving four categories: SISD (single instruction, single data), SIMD (single instruction, multiple data), MISD (multiple instruction, single data) and MIMD (multiple instruction, multiple data). This construction classifies architectures by their parallelism at the instruction and data level.

(b) One example for each parallel processor architecture: SISD — a conventional single-core uniprocessor (e.g., a basic scalar CPU); SIMD — a vector processor or GPU (e.g., array processor, graphics processing unit); MISD — a systolic array or a fault-tolerant system with replicated processing on the same data (rarely used; e.g., a pipeline of processors fed the same data stream); MIMD — a multicore processor or a cluster of workstations (e.g., Intel Core i7 multicore, distributed-memory multiprocessor).

**Unofficial marking guide:**

- *1 mark: stating Flynn's taxonomy based on instruction and data streams*
- *1 mark: naming the four categories (SISD, SIMD, MISD, MIMD) with correct meaning*
- *1 mark: correct example for SISD and SIMD*
- *1 mark: correct example for MISD*
- *1 mark: correct example for MIMD*

*Model confidence: high.*

*Model note: Assumes the classification intended is Flynn's taxonomy, which is standard for this topic. MISD examples are uncommon; a systolic array or replicated pipeline is acceptable.*

## QUESTION A2 (5 marks)

*Topic: Computer generations and history*

*Source: CSC429 (16).pdf, page(s) 2*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) The fourth computer generation (roughly 1971–1980s) was defined by two major technologies:

Technology 1 — Microprocessors (VLSI/LSI integration): Large-scale and very-large-scale integration allowed an entire CPU to be fabricated on a single silicon chip (e.g., Intel 4004, then 8080/8086). Impact on the modern computer: this made computers small, cheap, reliable and low-power, enabling the personal computer (PC) revolution and embedding processors in everyday devices. It is the direct ancestor of today's CPUs, and the continuing trend of packing more transistors onto a chip (Moore's Law) underpins modern desktops, laptops, smartphones and embedded systems.

Technology 2 — Semiconductor memory (RAM/ROM) and the GUI/microcomputer operating environment: fourth-generation machines replaced magnetic-core memory with semiconductor RAM and ROM, and introduced graphical user interfaces and user-friendly operating systems on microcomputers. Impact on the modern computer: fast, dense, inexpensive semiconductor memory is the basis of today's main memory hierarchy, and the GUI plus interactive OS established the way users interact with modern computers (windows, icons, mouse). Together these technologies made computing personal, affordable and accessible.

(b) One everyday example of artificial intelligence: virtual assistants/voice assistants on smartphones (e.g., Siri, Google Assistant, Alexa), which use speech recognition and natural language processing to understand spoken requests and respond — used daily for setting alarms, searching, and controlling smart devices. (Other acceptable examples: recommendation systems on streaming/shopping sites, spam filters, face unlock, navigation route prediction.)

**Unofficial marking guide:**

- *1 mark: correctly identifies a fourth-generation technology (e.g., microprocessor/VLSI) with brief description*
- *1 mark: explains its impact on the modern computer*

© Hak Cipta Universiti Teknologi MARA 

*• 1 mark: correctly identifies a second fourth-generation technology (e.g., semiconductor memory or GUI/microcomputer OS) with brief description*

*• 1 mark: explains its impact on the modern computer*

*• 1 mark: gives one valid everyday AI example with a brief explanation of its use*

*Model confidence: high.*

*Model note: Question text has OCR error 'A1' which is read as 'AI' (artificial intelligence). Generation boundaries vary slightly by textbook; answer uses the standard 1971–1980s microprocessor/VLSI definition of the fourth generation.*

## QUESTION A3 (5 marks)

*Topic: Data representation and number systems*

Answer: 144010 = 101101000002

*• 1 mark for identifying the target base.*

*• 1 mark for a correct repeated-division or place-value method.*

*• 1 mark for the correct digit sequence.*

*• 1 mark for the correct base subscript.*

*• 1 mark for a complete, consistent workflow.*

## QUESTION A4 (5 marks)

*Topic: Instruction formats and addressing modes*

*Source: CSC429 (1).pdf, page(s) 5*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the effective address of the operand itself, so the operand is fetched directly from memory location A — the address field points straight at the operand, with no intermediate pointer.

(b) How it is used: the CPU decodes the instruction, takes the address field A as the effective address (EA = A), and issues a memory read to that location. The operand stored at memory address A is then loaded into the CPU (e.g. into a register or the accumulator) for the operation. In the figure, the instruction's Address A field is used as the memory address, and the memory at that address holds the operand. No further address calculation or indirection is performed; the address field is used unchanged as the operand's location.

(c) Advantage: it is simple and fast — only one memory access is needed to fetch the operand (no extra indirection), and no address computation is required. Disadvantage: the address field must be large enough to hold a full memory address, limiting the range of addressable memory for a fixed instruction length; it is also inflexible (the address is fixed in the instruction, so it cannot easily support relocation or pointer-based structures).

**Unofficial marking guide:**

*• 1 mark: correctly labels the technique as direct addressing (accept absolute addressing).*

*• 1 mark: states that the address field A is used directly as the effective address (EA = A).*

*• 1 mark: explains the operand is fetched from memory at address A in a single memory access (no indirection).*

*• 1 mark: gives one valid advantage (e.g. simple/fast, single memory access, no address calculation).*

*• 1 mark: gives one valid disadvantage (e.g. limited addressable range due to address field size, inflexibility/no relocation or pointer support).*

*Model confidence: high.*

*Model note: Figure 2 is inferred from the extracted text: an instruction with Opcode and Address A, and memory containing the operand at that address. This layout corresponds to direct addressing; if the figure actually shows the address field pointing to a memory word that itself contains the operand's address, the answer would instead be indirect addressing. The subpart (b) reference to '9(a)' is treated as a typo for 'a)'.*

## QUESTION A5 (10 marks)

*Topic: Computer architecture and CPU organisation*

*Source: CSC429 (7).pdf, page(s) 3*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) A computer bus is a shared set of parallel communication lines (address, data and control) that carries information between the CPU, memory and I/O devices. The main bus types and their functions are:

• Address bus: a unidirectional set of lines that carries the address of the memory location or I/O port the

© Hak Cipta Universiti Teknologi MARA 

CPU wishes to access. Its width determines the maximum addressable memory (e.g. n lines → 2^n locations).

• Data bus: a bidirectional set of lines that carries the actual data (instructions or operands) between the CPU, memory and peripherals. Its width determines the word size and hence how many bits can be transferred per bus cycle.

• Control bus: a collection of individual control lines that coordinates and synchronises bus operations, carrying signals such as memory read/write, I/O read/write, interrupt request, bus request/grant, clock and reset.

• System (internal/processor) bus: connects the CPU to main memory and the cache/other high-speed units inside the computer; it is fast and short.

• Expansion (external/I/O) bus: connects the system bus to slower peripheral devices and expansion slots (e.g. PCI, USB), allowing I/O devices to be added; it is typically slower and may use a bridge to the system bus.

(Where a syllabus lists backplane, memory, I/O or local buses, the same functional descriptions apply: memory bus carries CPU–memory traffic, I/O bus carries CPU–peripheral traffic, backplane bus connects boards in a chassis.)

(b) Synchronous bus timing uses a common clock so that all bus events occur at fixed, known instants. Two benefits are:

1. Simplicity and low cost: because every device derives its timing from the same clock, no complex handshake (request/acknowledge) circuitry is needed; the bus protocol and control logic are simple and cheap to implement.

2. Speed/predictability: transfers complete in a fixed, known number of clock cycles, so the bus can run fast and timing is easy to predict and design for, giving high throughput for devices that can keep up with the clock.

(Other acceptable benefits: easier to design and debug, deterministic timing, efficient for closely matched fast devices.)

**Unofficial marking guide:**

*• 1 mark: correct function of address bus (carries address, unidirectional, width sets addressable memory)*
*• 1 mark: correct function of data bus (carries data, bidirectional, width sets word size)*
*• 1 mark: correct function of control bus (carries read/write, interrupt, clock, etc. to coordinate transfers)*
*• 1 mark: correct function of system/internal bus (connects CPU to memory/cache, fast)*
*• 1 mark: correct function of expansion/I/O bus (connects peripherals/expansion slots, slower, via bridge)*
*• 1 mark: any further correct bus type (backplane/memory/local) with correct function, or coherent overall description of bus structure*
*• 2 marks: first benefit of synchronous timing (e.g. simplicity/low cost, no handshake) with explanation*
*• 2 marks: second benefit (e.g. speed/predictable fixed-cycle timing) with explanation*
*Model confidence: high.*
*Model note: Question text does not enumerate specific bus types; answer assumes the standard address/data/control plus system and expansion bus set. If the course lists a different fixed set (e.g. backplane, memory, I/O), map the same functions to those names. No figure required.*

## QUESTION A6 (10 marks)

*Topic: Computer architecture and CPU organisation*

*Source: CSC429 (15).pdf, page(s) 3*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) Phases in the machine cycle [4 marks]

The machine cycle (also called the instruction cycle) consists of the following phases:

1. Fetch — the next instruction is fetched from main memory into the CPU, using the Program Counter (PC) to supply the address and the Memory Address Register (MAR) to hold it; the instruction is placed in the Memory Data Register (MDR) and then the Instruction Register (IR).
2. Decode — the Control Unit interprets the opcode in the IR and determines what operation is required and which operands are involved.
3. Execute — the required operation is carried out by the ALU and/or other units (arithmetic/logic operation, data movement, branch, etc.).
4. Store / Write-back (and interrupt check) — the result is written back to a register or to main memory, and the cycle checks for pending interrupts before returning to Fetch for the next instruction.

© Hak Cipta Universiti Teknologi MARA 

(b) Function of the different CPU components in the machine cycle [4 marks]
- Program Counter (PC): holds the address of the next instruction to be fetched; incremented during/after fetch so the next instruction is located.
- Memory Address Register (MAR): holds the address of the memory location to be accessed (instruction fetch or data read/write); connected to the address bus.
- Memory Data Register (MDR): buffers data/instructions passing between the CPU and main memory (holds the fetched instruction or the operand/result being transferred).
- Instruction Register (IR): holds the instruction currently being executed so the Control Unit can decode it.
- Control Unit (CU): decodes the opcode and generates the timing and control signals that coordinate all other components through the phases of the cycle.
- Arithmetic Logic Unit (ALU): performs arithmetic and logical operations during the execute phase and produces results/flags.
- Registers/Accumulator and general-purpose registers: hold operands and intermediate/result values during execution and write-back.
- Internal buses: carry data and control signals between these units.

(c) What happens if access to main memory is blocked [2 marks]
The Fetch phase cannot complete: the CPU cannot read the next instruction into the IR (nor read/write operands). Since the Control Unit has no instruction to decode, the machine cycle stalls/halts — the CPU is idle (or, if an interrupt/timeout mechanism exists, an error or memory-fault exception is raised). No useful instruction can be executed until memory access is restored, so the system effectively freezes or must rely on cache/interrupt handling.

**Unofficial marking guide:**

- *1 mark: Fetch phase correctly identified*
- *1 mark: Decode phase correctly identified*
- *1 mark: Execute phase correctly identified*
- *1 mark: Store/write-back (or interrupt check) phase correctly identified*
- *1 mark: PC and MAR roles in fetch explained*
- *1 mark: MDR and IR roles explained*
- *1 mark: Control Unit decode/control-signal role explained*
- *1 mark: ALU (and registers) role in execute/write-back explained*
- *1 mark: States fetch cannot complete / no instruction reaches IR*
- *1 mark: States CPU stalls/halts or raises a fault until memory access is restored*

*Model confidence: high.*

*Model note: Question text is clear; no figure required. Part (a) mark allocation assumes four named phases (fetch, decode, execute, store/interrupt-check); some texts merge store into execute, in which case award the fourth mark for the interrupt check or write-back stage. Part (b) awards marks for correct component-function pairs rather than requiring every listed component.*

## QUESTION A7 (10 marks)

*Topic: Memory hierarchy and cache*

*Source: CSC429 (20).pdf, page(s) 3*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) Access methods matched to scenarios (OCR-corrected interpretation):
- Reading a .c file from a folder → Sequential access (the file is read from start to end, byte by byte).
- Accessing a specific file block using an index in a large file → Direct/Random access (the index gives the block address, allowing direct jump to that block).
- Streaming files line by line → Sequential access (data is processed in order, one line after another).
- Jumping to a specific frame or timestamp in a video → Direct/Random access (the timestamp/frame number provides an address for direct positioning).

(b) Ranking from most to least storage capacity: Tape > Hard drive > Main memory > L3 Cache > L1 Cache > Register.

(c) Volatile memory loses its stored contents when power is removed. It is used for temporary storage such as main memory (RAM) and cache, which hold programs and data currently in use. Non-volatile memory (e.g., hard drive, tape, ROM) retains data without power.

**Unofficial marking guide:**

- *1 mark: Correctly identifies sequential access for reading a .c file from a folder.*
- *1 mark: Correctly identifies direct/random access for accessing a specific file block using an index.*

© Hak Cipta Universiti Teknologi MARA 

*• 1 mark: Correctly identifies sequential access for streaming files line by line.*
*• 1 mark: Correctly identifies direct/random access for jumping to a specific frame or timestamp in a video.*
*• 1 mark: Provides a brief justification for each access method (total 5 marks for part a).*
*• 1 mark: Correctly places Tape as the largest capacity.*
*• 1 mark: Correctly orders Hard drive > Main memory > L3 Cache > L1 Cache > Register (3 marks for part b).*
*• 1 mark: Defines volatile memory as losing contents when power is removed.*
*• 1 mark: Gives an example (e.g., RAM, cache) and/or contrasts with non-volatile memory (2 marks for part c).*
*Model confidence: medium.*
*Model note: The question text contains OCR errors; the scenarios were conservatively interpreted as: reading a .c file from a folder, accessing a specific file block using an index, streaming files line by line, and jumping to a specific frame/timestamp in a video. The access method table headers were assumed to be 'Access method' and 'Scenario'. Part (b) ranking assumes typical capacity hierarchy; actual capacities may vary but the order is standard.*

## QUESTION A8 (10 marks)

*Topic: Input/output and interrupts*

*Source: CSC429 (3).pdf, page(s) 4*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) Two types of characters provided by a keyboard [4 marks]

A keyboard does not send a single uniform kind of character; the characters it provides fall into two broad types:

1. Printable (graphic/visible) characters: These are characters that have a visible glyph and are meant to be displayed or printed, e.g. letters (A–Z, a–z), digits (0–9), punctuation marks (., ; ? !), and symbols (@, #, $, %). When a key for a printable character is pressed, the keyboard's encoder generates the corresponding character code (e.g. ASCII) which the I/O module passes to the processor so the character can be echoed to the screen or stored in memory.

2. Control (non-printable) characters: These are characters that do not produce a visible symbol but instead cause an action or control function, e.g. Enter/Return (carriage return/line feed), Tab, Backspace, Delete, Escape, and cursor/arrow keys. They are represented by control codes (e.g. ASCII control codes 0–31) and are interpreted by the operating system or application to perform an operation such as moving the cursor, starting a new line, or deleting a character.

(An acceptable alternative pairing is alphanumeric characters versus special/symbol characters, provided each is explained and an example given.)

(b) Three categories of input/output module [6 marks]

An I/O module is the hardware that connects the processor/memory to peripheral devices. Its main categories, with examples, are:

1. Human-readable / human-interaction modules: These interface the computer to a human operator, converting human actions into machine signals and machine output into a form people can perceive. Example: keyboard and display (monitor), or a mouse/printer.

2. Machine-readable modules: These interface the computer to equipment or media that store/read data in machine-usable form, without a human directly reading the data. Example: disk drive (magnetic/optical disk controller), magnetic tape, or a sensor/actuator interface.

3. Communication modules: These allow the computer to exchange data with other computers or with remote devices over a network or communication link. Example: network interface card (NIC)/modem, or a serial communication port.

Each category is distinguished by the type of device it serves (human, machine/storage, or communication), and the I/O module provides the necessary control, buffering, and data-format conversion for that device.

**Unofficial marking guide:**

*• 1 mark: correctly naming the first type of keyboard character (e.g. printable/graphic characters)*
*• 1 mark: explaining the first type with an example (letters, digits, symbols)*
*• 1 mark: correctly naming the second type of keyboard character (e.g. control/non-printable characters)*
*• 1 mark: explaining the second type with an example (Enter, Tab, Backspace, arrow keys)*
*• 1 mark: correctly naming the first I/O module category (human-readable)*
*• 1 mark: giving an appropriate example for the first category (keyboard, display, mouse, printer)*
*• 1 mark: correctly naming the second I/O module category (machine-readable)*
*• 1 mark: giving an appropriate example for the second category (disk drive, tape, sensor)*

© Hak Cipta Universiti Teknologi MARA 

*• 1 mark: correctly naming the third I/O module category (communication)*

*• 1 mark: giving an appropriate example for the third category (NIC, modem, serial port)*

*Model confidence: high.*

*Model note: The question text does not specify which character classification is intended; the answer uses printable vs control characters, which is the standard interpretation, and notes an acceptable alternative (alphanumeric vs special symbols). The I/O module categories follow the standard Stallings classification (human-readable, machine-readable, communication). No figure or table is required.*

## QUESTION A9 (10 marks)

*Topic: Input/output and interrupts*

*Source: CSC429 (10).pdf, page(s) 5*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

(a) Three types of interfaces to external devices [6 marks]

1. Programmed I/O (polled I/O) interface: The CPU directly controls the transfer and repeatedly checks (polls) the device's status register to see whether it is ready. Data passes through the CPU (e.g., via an accumulator) between the device and memory. It is simple and needs no special interrupt hardware, but it wastes CPU time busy-waiting, so it suits slow or infrequently used devices.

2. Interrupt-driven I/O interface: The device signals the CPU with an interrupt request when it is ready to transfer or has completed a transfer. The CPU suspends its current program, runs an interrupt service routine (ISR) to service the device, then resumes. This frees the CPU from polling, giving better utilisation, though each transfer still involves CPU intervention and interrupt overhead.

3. Direct Memory Access (DMA) interface: A dedicated DMA controller takes over the bus and transfers blocks of data directly between the device and main memory without CPU involvement per word. The CPU programs the controller (start address, count, direction) and is interrupted only once per block. This gives high throughput for fast devices such as disks and network cards, at the cost of extra hardware and bus arbitration (cycle stealing).

(Other acceptable interfaces: memory-mapped I/O vs isolated/port-mapped I/O, serial vs parallel interfaces, or synchronous vs asynchronous interfaces, if described as interfaces to external devices.)

(b) Two types of interrupt [4 marks]

1. Maskable interrupt: An interrupt that can be disabled (masked) by the CPU, typically by clearing a bit in the interrupt-enable register or by the interrupt mask in the status register. It is used for ordinary I/O devices whose service can be delayed; the CPU can ignore it temporarily while executing a critical section. Example: a keyboard or timer interrupt.

2. Non-maskable interrupt (NMI): An interrupt that cannot be disabled by the normal masking mechanism. It is reserved for high-priority, urgent events such as power failure, memory parity error or a hardware fault, and is always recognised so the CPU can take immediate recovery action.

(Alternative acceptable pairing: hardware/external interrupt vs software/internal interrupt, or vectored vs non-vectored interrupt, if each is explained.)

**Unofficial marking guide:**

*• 1 mark: naming the first interface type (e.g., programmed/polled I/O)*

*• 1 mark: correct description of how the first interface operates (CPU polls status, data through CPU)*

*• 1 mark: naming the second interface type (e.g., interrupt-driven I/O)*

*• 1 mark: correct description of how the second interface operates (device raises interrupt, ISR services it)*

*• 1 mark: naming the third interface type (e.g., DMA)*

*• 1 mark: correct description of how the third interface operates (controller transfers block directly to/from memory, one interrupt per block)*

*• 1 mark: naming the first interrupt type (e.g., maskable)*

*• 1 mark: explanation of the first interrupt type (can be disabled/masked, used for ordinary I/O)*

*• 1 mark: naming the second interrupt type (e.g., non-maskable)*

*• 1 mark: explanation of the second interrupt type (cannot be masked, used for urgent/fatal events)*

*Model confidence: high.*

*Model note: Question text is complete; no figure or table is required. The paper's slot label (A9) differs from the question id (A8); assumed the stated 10-mark total (6 + 4) is correct. Alternative valid interface/interrupt pairings are accepted as indicated in the answer.*

© Hak Cipta Universiti Teknologi MARA 

**QUESTION A10** **(10 marks)**

*Topic: Data representation and number systems*

a) Answer: left 30 -> 0 0011110; right -25 -> 1 0011001; 30 - (-25) = 55 -> sign-magnitude 0 0110111

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left 30 -> 1's 00011110, 2's 00011110; right -25 -> 1's 11100110, 2's 11100111; 30 - (-25) = 55 -> 1's 00110111, 2's 00110111

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

© Hak Cipta Universiti Teknologi MARA

# PART B — MARKING GUIDE

## QUESTION B1 (20 marks)

*Topic: Assembly-language programming*

*Source: CSC429 (17).pdf, page(s) 5*

**Unofficial model answer — generated by an AI model, not an official UTM marking scheme.**

Assumption on the equation (OCR correction): Figure 2 reads y = A(A + 2B/C-D 2). The only consistent reading with the available instructions (no exponentiation) is y = A × (A + 2B/C − D²), i.e. the trailing 2 is a superscript square on D. This is assumed throughout; if the figure shows a different expression, the operand order changes but the method is identical.

Sub-expression plan (common to all three programs): compute T1 = 2B/C, T2 = D×D, T3 = A + T1 − T2, then y = A × T3. A, B, C, D are read-only; x, y, z are temporaries; final result in y.

Zero-address program (stack machine):

```
PUSH A        ; stack: A
    PUSH B        ; stack: A, B
    PUSH B        ; stack: A, B, B
    ADD           ; stack: A, 2B       (B+B)
    PUSH C        ; stack: A, 2B, C
    DIV           ; stack: A, 2B/C
    ADD           ; stack: A + 2B/C
    PUSH D        ; stack: (A+2B/C), D
    PUSH D        ; stack: (A+2B/C), D, D
    MUL           ; stack: (A+2B/C), D*D
    SUB           ; stack: A + 2B/C - D*D
    PUSH A        ; stack: (A+2B/C-D*D), A
    MUL           ; stack: A*(A+2B/C-D*D)
    POP y         ; store final answer in y
```

One-address program (accumulator machine):

```
LOAD B        ; ACC = B
    ADD B         ; ACC = 2B
    DIV C         ; ACC = 2B/C
    ADD A         ; ACC = A + 2B/C
    STOR x        ; x = A + 2B/C   (save partial result)
    LOAD D        ; ACC = D
    MUL D         ; ACC = D*D
    STOR z        ; z = D*D
    LOAD x        ; ACC = A + 2B/C
    SUB z         ; ACC = A + 2B/C - D*D
    MUL A         ; ACC = A*(A + 2B/C - D*D)
    STOR y        ; y = final answer
```

Three-address program (register/operand form ADD(p,q,r) meaning p = q op r):

```
ADD (x, B, B)     ; x = 2B
    DIV (x, x, C)     ; x = 2B/C
    ADD (x, A, x)     ; x = A + 2B/C
    MUL (z, D, D)     ; z = D*D
    SUB (x, x, z)     ; x = A + 2B/C - D*D
    MUL (y, A, x)     ; y = A*(A + 2B/C - D*D)  final answer in y
```

Comment on correctness: In every version the read-only registers A, B, C, D are only ever read as source operands, the temporaries x and z hold intermediate values, and the final result is written to y, satisfying the stated constraints.

**Unofficial marking guide:**

*• 4 marks: zero-address program — correct use of PUSH/POP with stack semantics, correct operand order for DIV and SUB, and final POP into y*

*• 4 marks: one-address program — correct LOAD/STOR/ADD/SUB/MUL/DIV sequence, correct use of temporaries x and z, final STOR y*

© Hak Cipta Universiti Teknologi MARA 

*• 4 marks: three-address program — correct ADD/DIV/MUL/SUB with (p,q,r) destination-first convention, correct final MUL into y*

*• 4 marks: correct evaluation of the expression (2B/C, D*D, A + 2B/C − D*D, then multiply by A) in all three programs*

*• 2 marks: comments and correct assembly notation throughout, with A, B, C, D unmodified and final answer in y*

*• 2 marks: overall clarity, consistent register/temporary usage and no redundant or missing operations*

*Model confidence: medium.*

*Model note: The equation in Figure 2 is OCR-ambiguous ('D 2'); assumed to be D squared since no exponent instruction exists. The three-address table shows 'MUL (p.q.r)' — read as MUL(p,q,r). The (p,q,r) convention is assumed to be destination-first (p = q op r); if the lecturer uses source-first ordering, swap the operand positions accordingly. Figure 2 itself was not available beyond the extracted text.*

*END OF MARKING SCHEME*

© Hak Cipta Universiti Teknologi MARA